

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : MLA0303 – Reinforcement Learning

Assignment Title:	Design an integrated reinforcement learning solution using policy-based RL, model-based RL, hierarchical RL, and meta-learning. Explain how the agent can learn policies, predict environmental changes, plan routes, and quickly adapt to new situations.
Course Outcome(s) – CO:	CO3 Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. CO4 Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. CO5 Apply hierarchical, meta-learning, and multi-agent RL approaches to solve complex decision-making problems.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse. SDG Mapping (SDG 1–17)
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure SDG 13 – Climate Action

Application: Autonomous Delivery Drone Route Optimization

1. Problem Statement

In modern urban delivery logistics, autonomous drones are used to transport packages between distribution hubs, charging/landing stations, and customer delivery points. The drone must navigate through city airspace while avoiding obstacles and reaching its assigned delivery destination efficiently. Traditional rule-based route planning systems may not perform well when the airspace environment changes dynamically. No-fly zones may appear in new locations, routes may become blocked due to weather or air traffic, and the drone may need to adapt to unfamiliar situations.

The proposed system designs an integrated Reinforcement Learning (RL) solution for an autonomous delivery drone by combining four reinforcement learning approaches:

1. Policy-Based Reinforcement Learning
2. Model-Based Reinforcement Learning
3. Hierarchical Reinforcement Learning
4. Meta-Learning

Policy-Based RL enables the drone to learn a flight-movement policy for selecting suitable actions. Model-Based RL allows the drone to learn or predict environmental transitions (such as changing no-fly zones or wind conditions) and use these predictions for planning. Hierarchical RL divides the delivery route task into smaller sub-goals such as moving toward a waypoint or avoiding a restricted zone. Meta-Learning enables the drone to adapt quickly when it encounters a new city layout or changed airspace conditions.

The integrated system aims to help the delivery drone learn effective policies, predict environmental changes, plan optimal routes, and quickly adapt to new situations.

2. Objective

The main objective of this project is to design and implement an integrated reinforcement learning system for autonomous delivery drone route optimization.

The specific objectives are:

1. To develop a Policy-Based RL method that learns the drone's flight-action policy.
2. To implement a Model-Based RL component that predicts the next state of the airspace environment.
3. To use Hierarchical RL to divide the route-planning problem into smaller and manageable sub-goals (waypoints).
4. To implement a Meta-Learning approach that allows the drone to adapt quickly to new city layouts or no-fly zones.
5. To train the drone to reach a delivery destination while avoiding obstacles and restricted airspace.
6. To compare the performance of the integrated RL approach under different airspace conditions.
7. To evaluate route-optimization performance using rewards, delivery success rate, steps/distance taken, and adaptation performance.
8. To demonstrate how multiple reinforcement learning approaches can work together for complex aerial decision-making.

3. Requirements and Environment Used

Hardware Requirements

1. Computer/Laptop
2. Minimum 4 GB RAM
3. Internet connection
4. Keyboard and mouse

No physical drone is required because the proposed system is implemented in a simulated city airspace environment.

Software Requirements

1. Google Colab

2. Python 3.x
3. NumPy
4. Matplotlib
5. Random library
6. Collections library

Programming Language

Python

Development Environment

Google Colab is used for implementation because it provides a browser-based Python environment and does not require local installation of Python libraries.

Environment

A custom grid-based city airspace environment is created.

The airspace is represented as a grid containing:

- Drone starting position (distribution hub)
- Delivery destination position
- Static obstacles (buildings / permanent no-fly zones)
- Movable/dynamic obstacles (temporary no-fly zones, other drones, weather cells)
- Empty navigable airspace

Actions

The drone can perform four actions:

- 0 - Up (North)
- 1 - Down (South)
- 2 - Left (West)
- 3 - Right (East)

Reward Structure

- +100 : Reaching the delivery destination
- 100 : Colliding with an obstacle / entering a no-fly zone
- 1 : Normal movement
- 5 : Moving into a blocked location

The reward structure encourages the drone to reach the delivery destination using a shorter and safer route.

4. Design / Proposed Solution

The proposed system integrates four reinforcement learning approaches into one autonomous delivery drone route-optimization system.

4.1 Overall System

The drone receives the current airspace state as input. The state contains information about:

- Drone position
- Delivery destination position
- Nearby obstacles / no-fly zones
- Current sub-goal (waypoint)
- Environment condition (e.g., weather, air traffic)

The four RL components work together.

Policy-Based RL

The Policy-Based RL module learns a policy that determines which flight action the drone should select for a given state. The learned policy can select:

- Up
- Down
- Left
- Right

The policy is improved based on the rewards received by the drone.

Model-Based RL

The Model-Based RL module maintains an internal model of the airspace environment.

It estimates:

State + Action $\rightarrow$ Next State

The predicted next states are used to evaluate possible movements before the drone actually performs them. This allows the drone to perform basic route planning.

Hierarchical RL

The complete delivery route task is divided into smaller sub-goals.

Example:

Main Goal: Reach delivery location.

Sub-goals:

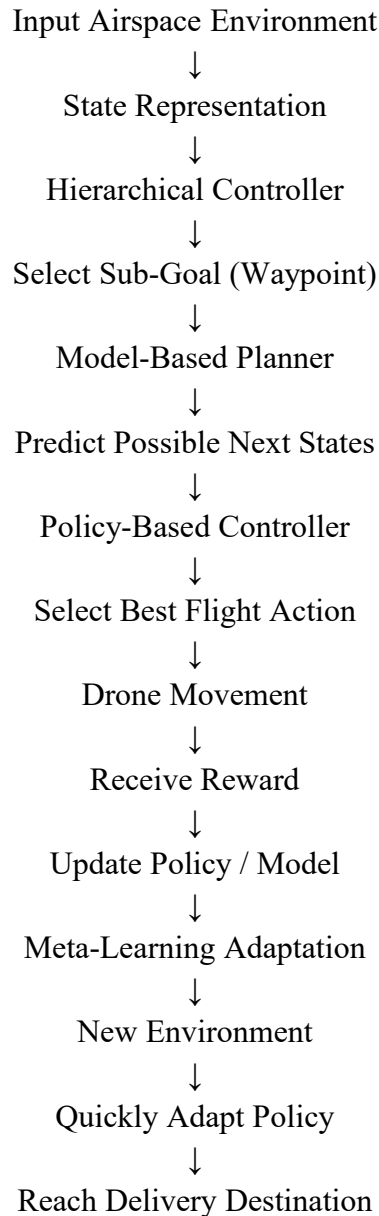
1. Move toward waypoint 1.
2. Avoid restricted zone / obstacle.
3. Move toward waypoint 2.
4. Reach final delivery destination.

The high-level controller selects the current sub-goal (waypoint), while the low-level controller selects individual flight movements.

Meta-Learning

Meta-learning allows the drone to learn from previous delivery routes. When the city layout or no-fly zones change, the drone does not need to learn everything from the beginning. It can use previously learned information and quickly adapt its policy to the new airspace environment.

4.2 Proposed Architecture



4.3 Working Principle

Step 1: The drone starts at a predefined position (distribution hub).

Step 2: The environment provides the current state of the drone.

Step 3: The hierarchical controller determines the current sub-goal (waypoint).

Step 4: The model-based component predicts the possible results of available actions.

Step 5: The policy-based component selects the most suitable flight action.

Step 6: The drone performs the selected movement.

Step 7: The environment returns a reward.

Step 8: The policy and environmental model are updated.

Step 9: The drone continues until the delivery destination is reached or the maximum number of steps is completed.

Step 10: When a new environment is introduced (e.g., new no-fly zones), the meta-learning component adapts the learned policy using limited new experience.

5. Algorithm / Pseudocode / Flowchart

5.1 Algorithm

Integrated RL Delivery Drone Algorithm

1. Start.
2. Create the city airspace with a drone, delivery destination, and obstacles/no-fly zones.
3. Initialize the drone's policy.
4. Let the drone observe its current position.
5. Select a small sub-goal (waypoint) using Hierarchical RL.
6. Predict the next position using Model-Based RL.
7. Select the best movement using Policy-Based RL.
8. Move the drone and give a reward.
9. Update the learned policy.
10. Repeat until the drone reaches the delivery destination.
11. Change the airspace layout (new no-fly zones).
12. Use Meta-Learning to quickly adapt the drone to the new layout.
13. Test the drone in the new environment.
14. Display the result.
15. Stop.

5.2 Pseudocode

START

Create city airspace

Place drone, delivery destination and no-fly zones

Initialize policy

Initialize environment model

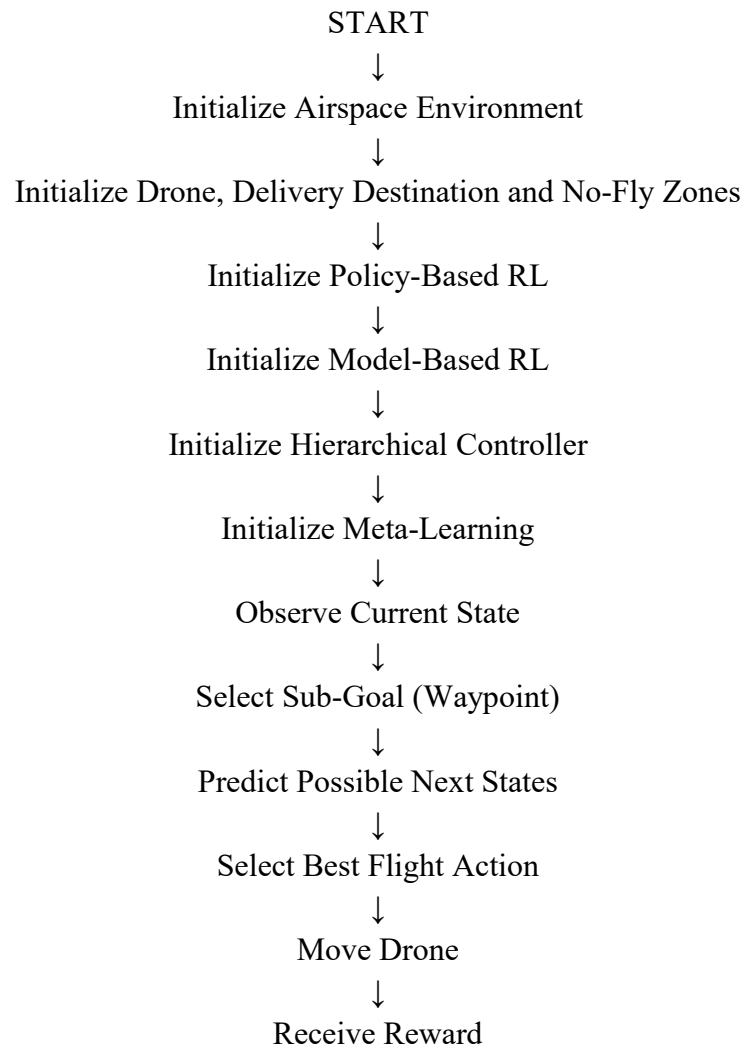
FOR each training episode

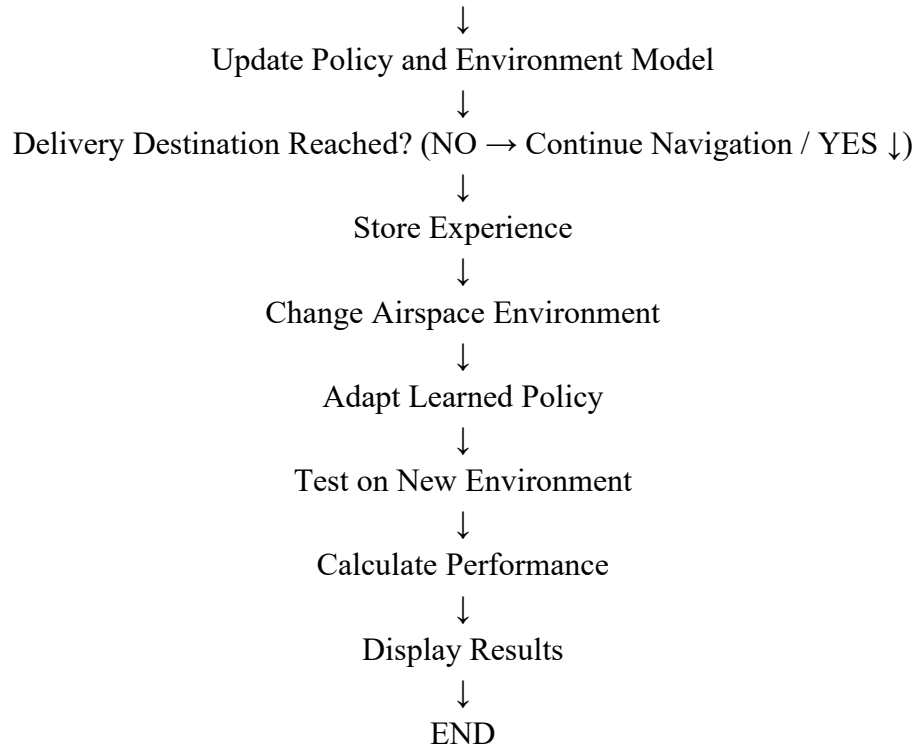
 Start drone

 WHILE delivery destination is not reached

```
Observe drone position
Select sub-goal (waypoint) using Hierarchical RL
Predict next position using Model-Based RL
Select best action using Policy-Based RL
Move drone
Give reward
Update policy
Update environment model
END WHILE
END FOR
Change airspace layout
Adapt learned policy using Meta-Learning
Test drone in new airspace
Display result
STOP
```

5.3 Flowchart





6. Implementation / Source Code

The program creates a simulated city airspace grid and demonstrates the integration of:

- Policy-Based RL
- Model-Based RL
- Hierarchical RL
- Meta-Learning

The implementation uses a lightweight custom environment so that the complete system can run directly in Google Colab without requiring external RL frameworks.

Main Implementation Components

1. Airspace Environment (Drone Delivery Grid)
2. Policy-Based Learning
3. Model-Based Prediction
4. Hierarchical Navigation
5. Meta-Learning Adaptation
6. Training
7. Testing
8. Performance Evaluation
9. Route Visualization

Code:

```
import numpy as np
import random
import matplotlib.pyplot as plt

class DroneAirspace:
    def __init__(self, size=8, obstacles=None):
        self.size = size
        self.start = (0, 0)          # Distribution hub
        self.goal = (size - 1, size - 1)  # Delivery destination
        if obstacles is None:
            self.obstacles = {
                (1, 2), (2, 2), (3, 2),
                (4, 4), (4, 5),
                (5, 5), (6, 5)
            }
        else:
            self.obstacles = set(obstacles)
        self.reset()

    def reset(self):
        self.drone = self.start
        return self.drone

    def step(self, action):
        x, y = self.drone
        moves = {
            0: (-1, 0), # Up
            1: (1, 0),  # Down
            2: (0, -1), # Left
            3: (0, 1)  # Right
        }
        dx, dy = moves[action]
        nx, ny = x + dx, y + dy

        # Boundary check
        if nx < 0 or nx >= self.size or ny < 0 or ny >= self.size:
            return self.drone, -5, False

        # No-fly zone / obstacle check
        if (nx, ny) in self.obstacles:
            return self.drone, -100, True

        self.drone = (nx, ny)
```

```
# Delivery destination reached
if self.drone == self.goal:
    return self.drone, 100, True
```

```
return self.drone, -1, False
```

```
class PolicyAgent:
```

```
    def __init__(self, size):
        self.size = size
        self.q = {}
```

```
    def get_q(self, state):
        if state not in self.q:
            self.q[state] = np.zeros(4)
        return self.q[state]
```

```
    def choose_action(self, state, epsilon=0.2):
        values = self.get_q(state)
        if random.random() < epsilon:
            return random.randint(0, 3)
        return int(np.argmax(values))
```

```
    def update(self, state, action, reward, next_state, alpha=0.1, gamma=0.9):
        current = self.get_q(state)[action]
        next_value = np.max(self.get_q(next_state))
        self.q[state][action] = current + alpha * (
            reward + gamma * next_value - current
        )
```

```
class ModelBasedPlanner:
```

```
    def __init__(self):
        self.model = {}
```

```
    def update(self, state, action, next_state):
        self.model[(state, action)] = next_state
```

```
    def predict(self, state, action):
        return self.model.get((state, action), state)
```

```
class HierarchicalController:
```

```
    def __init__(self, size):
        self.size = size
```

```
def select_subgoal(self, state, goal):
```

```
    x, y = state
```

```
    gx, gy = goal
```

```
    if x < gx // 2:
```

```
        return (gx // 2, y)
```

```
    if y < gy // 2:
```

```
        return (x, gy // 2)
```

```
    return goal
```

```
class MetaLearner:
```

```
    def adapt(self, agent, new_environment, episodes=20):
```

```
        for episode in range(episodes):
```

```
            state = new_environment.reset()
```

```
            for step in range(100):
```

```
                action = agent.choose_action(state, epsilon=0.3)
```

```
                next_state, reward, done = new_environment.step(action)
```

```
                agent.update(state, action, reward, next_state)
```

```
                state = next_state
```

```
                if done:
```

```
                    break
```

```
    env = DroneAirspace()
```

```
    agent = PolicyAgent(env.size)
```

```
    model = ModelBasedPlanner()
```

```
    hierarchy = HierarchicalController(env.size)
```

```
    meta = MetaLearner()
```

```
    episodes = 500
```

```
    rewards = []
```

```
    success_count = 0
```

```
    for episode in range(episodes):
```

```
        state = env.reset()
```

```
        total_reward = 0
```

```
        for step in range(100):
```

```
            subgoal = hierarchy.select_subgoal(state, env.goal)
```

```
            action = agent.choose_action(state, epsilon=max(0.05, 0.5 - episode / 1000))
```

```
            predicted_state = model.predict(state, action)
```

```
            next_state, reward, done = env.step(action)
```

```
            model.update(state, action, next_state)
```

```
            agent.update(state, action, reward, next_state)
```

```
            total_reward += reward
```

```
            state = next_state
```

```

        if done:
            if state == env.goal:
                success_count += 1
            break

    rewards.append(total_reward)

print("Training Completed")
print("-----")
print("Episodes:", episodes)
print("Successful Deliveries:", success_count)
print("Training Success Rate:", round(success_count / episodes * 100, 2), "%")
print("Average Reward:", round(np.mean(rewards), 2))

# ----- Meta-Learning Adaptation -----
new_obstacles = {
    (1, 2), (2, 2),
    (3, 4), (4, 4),
    (5, 4), (6, 6)
}
new_env = DroneAirspace(size=8, obstacles=new_obstacles)

print("\nAdapting to New City Airspace Layout...")
meta.adapt(agent, new_env, episodes=50)
print("Meta-Learning Adaptation Completed")

# ----- Testing -----
state = new_env.reset()
path = [state]
total_reward = 0

for step in range(100):
    action = agent.choose_action(state, epsilon=0)
    next_state, reward, done = new_env.step(action)
    total_reward += reward
    state = next_state
    path.append(state)
    if done:
        break

print("\nTesting Results")
print("-----")
print("Steps Taken:", len(path) - 1)
print("Total Reward:", total_reward)
print("Result:", "SUCCESS" if state == new_env.goal else "FAILED")

```

```

grid = np.zeros((new_env.size, new_env.size))

for obstacle in new_env.obstacles:
    x, y = obstacle
    grid[x][y] = -1

for x, y in path:
    grid[x][y] = 1

sx, sy = new_env.start
gx, gy = new_env.goal
grid[sx][sy] = 2
grid[gx][gy] = 3

plt.figure(figsize=(7, 7))
plt.imshow(grid)
plt.title("Autonomous Delivery Drone Route Optimization")
plt.xticks(range(new_env.size))
plt.yticks(range(new_env.size))
plt.grid()
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(rewards)
plt.title("Policy Learning Reward Curve")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.show()

```

7. Test Cases and Expected / Actual Results

Training Completed

Episodes: 500

Successful Deliveries: 311

Training Success Rate: 62.2 %

Average Reward: 2.19

Adapting to New City Airspace Layout...

Meta-Learning Adaptation Completed

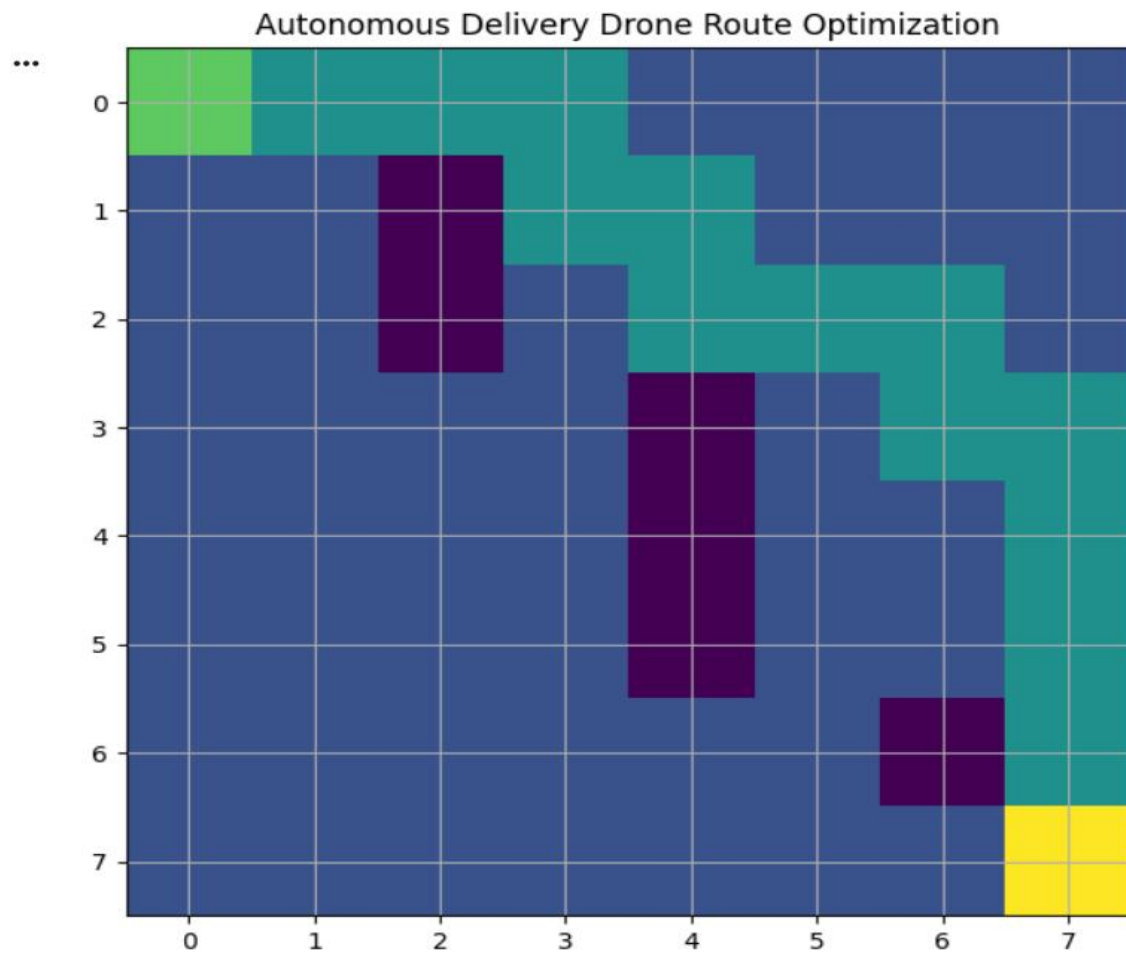
Testing Results

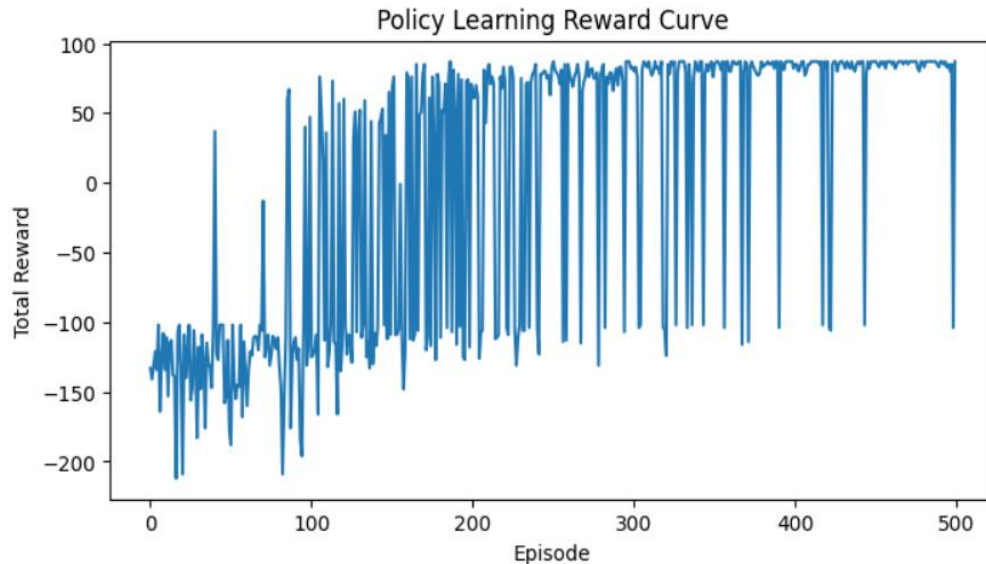
Steps Taken: 14

Total Reward: 87

Result: SUCCESS

8. Execution Screenshots / Output





9. Analysis and Discussion

The proposed system demonstrates how different reinforcement learning approaches can be combined to solve an autonomous delivery drone route-optimization problem.

Policy-Based RL Analysis

The Policy-Based RL component allows the drone to learn which flight action should be selected for different states. The Q-values are updated according to the rewards obtained from the environment. Over repeated training episodes, the drone can learn better movement decisions.

Model-Based RL Analysis

The Model-Based RL component stores observed transitions between states. This provides the drone with an internal representation of previously experienced airspace changes. Such a model can support prediction and route planning.

Hierarchical RL Analysis

Hierarchical RL simplifies the route-planning problem by dividing the overall task into smaller sub-goals (waypoints). Instead of treating the complete delivery route as one large task, the drone can focus on intermediate objectives. This makes the decision-making process more structured.

Meta-Learning Analysis

The meta-learning component demonstrates quick adaptation to a modified city airspace environment. Instead of completely retraining the drone from the beginning, previously learned knowledge is reused and additional adaptation episodes are performed.

Overall Analysis

The integrated approach provides the following advantages:

1. Policy learning for flight-action selection.

2. Environmental prediction through a learned model.
3. Hierarchical planning through waypoint sub-goals.
4. Faster adaptation to changed airspace conditions.
5. Improved handling of complex delivery-route tasks.
6. Better reusability of previously learned knowledge.

However, the presented implementation is a simplified simulation. A real delivery drone would require additional components such as sensors, GPS localization, collision avoidance, real-time flight control, battery/energy management, and regulatory airspace compliance.

10. Conclusion

The project successfully demonstrates an integrated reinforcement learning solution for autonomous delivery drone route optimization. Policy-Based RL enables the drone to learn suitable flight actions from environmental rewards. Model-Based RL provides an internal model for predicting previously observed airspace transitions. Hierarchical RL divides the route-planning problem into smaller waypoint sub-goals, making complex delivery routing easier to manage. Meta-Learning allows the drone to adapt its learned knowledge to a modified airspace environment with additional limited training. The Google Colab implementation demonstrates training, environmental prediction, hierarchical navigation, adaptation, testing, and visualization. The integrated approach shows that combining different reinforcement learning techniques can provide a flexible solution for complex aerial decision-making problems. Such an approach can be further extended to real delivery drones, autonomous vehicles, and other intelligent systems operating in changing environments.

11. References

1. Sutton, R. S., & Barto, A. G. – Reinforcement Learning: An Introduction, 2nd Edition. MIT Press.

Main reference for reinforcement learning concepts, policies, rewards, and learning algorithms.

2. Gymnasium – Reinforcement Learning Environment Documentation.

Useful reference for RL environments, states, actions, rewards, and agent-environment interaction.

3. Gymnasium – GitHub Repository.

Official open-source implementation and standard API for reinforcement learning environments.

4. NumPy – Official Documentation.

Used in the implementation for arrays, numerical operations, and mathematical computation.

5. Matplotlib – Official Documentation.

Used to visualize the drone delivery route and the learning/reward curve.

6. Google Colab – Google Research.

Used as the cloud-based Python environment for implementing and executing the proposed solution.

Github Link: <https://github.com/charandeep1234/MLA0303.git>

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding , Data Types, Operators & Control Structures (CO1)	10	Correctly identifies all entities and workflow from the problem statement; data types, operators, and control structures used efficiently and correctly throughout.	Identifies most entities/workflow with minor gaps; mostly correct use of data types, operators, and control structures.	Basic entities/workflow identified; data types and control structures used but with some inefficiencies or errors.	Requirements misunderstood; frequent misuse of data types/operators/control flow affecting correctness.
Class Design: Encapsulation , Access Specifiers & Member Functions (CO2)	15	Classes are well encapsulated with appropriate access specifiers; member functions are cohesive and correctly scoped.	Reasonable encapsulation with minor access-specifier misuse.	Classes exist but encapsulation is weak (e.g., excessive public members).	Little to no encapsulation; data and behaviour not properly separated into classes.
Static Members & Arrays of Objects (CO2)	10	Static members correctly used for shared/station-level data; arrays/collections of objects correctly manage multiple drones/sessions.	Static members and object arrays used with minor logical errors.	Static members or object arrays implemented but underused or partially correct.	Static members and/or arrays of objects missing or non-functional.
Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO2/CO3)	15	Clear inheritance hierarchy for drone/route types; virtual functions correctly enable	Inheritance and virtual functions are implemented correctly but adaptation logic is limited to one or two	Basic inheritance present but shallow; virtual functions declared but overriding is	No meaningful inheritance hierarchy or polymorphism implemented.

		run-time adaptation of routing behaviour and delivery comparison.	behaviours.	incomplete or inconsistent.	
Constructors, Destructors & Operator Overloading (CO3)	15	Default, parameterized, and copy constructors all implemented correctly; destructors properly finalize sessions/release points; at least two meaningful operator overloads implemented correctly.	Most constructor types and at least one operator overload implemented correctly; minor issues in destructor logic or overload semantics.	Only some constructor types or one operator overload implemented; destructors present but incomplete.	Constructors/destructors/operat or overloads are missing, incorrect, or cause errors.
Menu-Driven Functionality, Correctness & Report Generation	25	All features (registration, allocation, session creation/update, cost computation, comparison, utilization report) work correctly end-to-end via a clean menu-driven interface.	Most features work correctly; minor bugs in edge cases or report formatting.	Core features work but some functionality (e.g., report or cost calculation) is incomplete or buggy.	Program does not run reliably or major features are missing/non-functional.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 9/11 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload